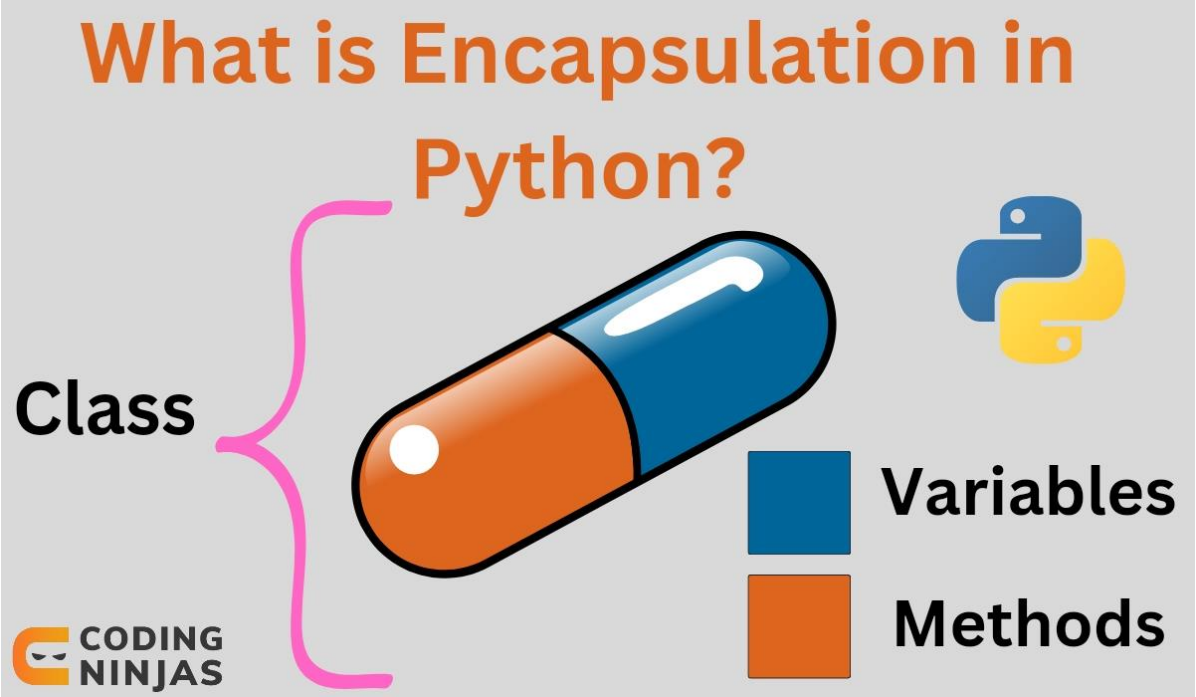
Encapsulation & Abstraction

Encapsulation
(Access via convention)

- Binding **data + methods** together
- Controlling access



Access Levels in Python (Convention)

Type	Syntax
Public	name
Protected	_name
Private	__name

Example

```python
class Bank:
    def __init__(self):
        self.name = "User"        # public
        self._balance = 5000      # protected
        self.__pin = 1234         # private

    def show(self):
        print(self.__pin)

b = Bank()
print(b.name)
print(b._balance)
# print(b.__pin) → error
b.show()
```

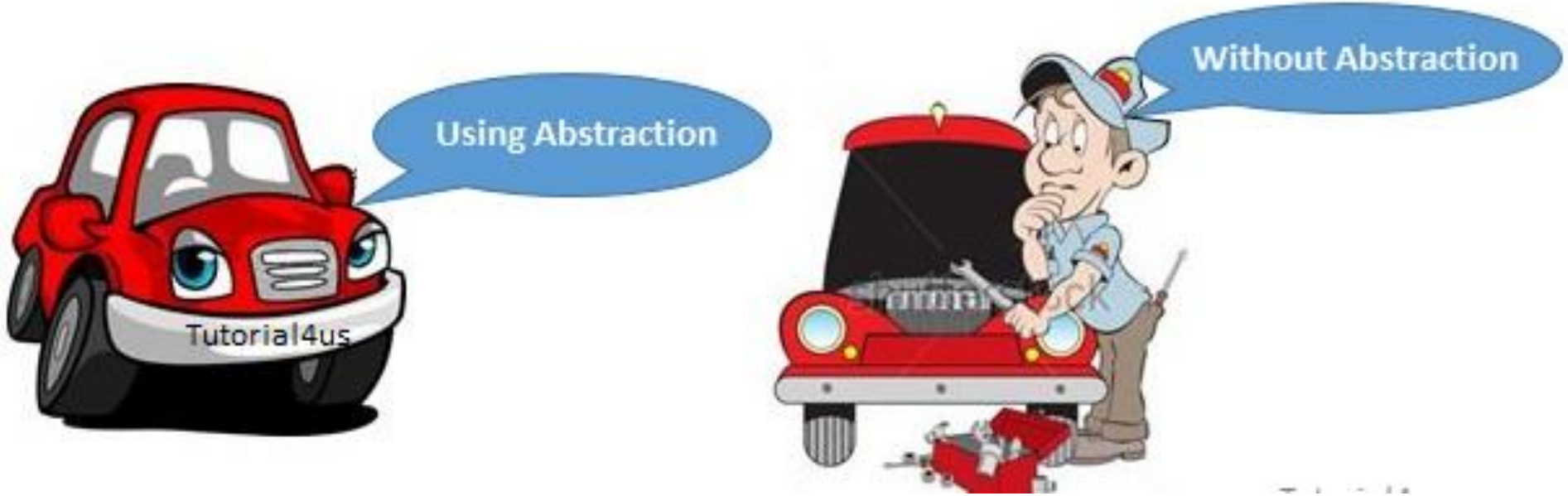
Why Encapsulation?

- Data safety
- Controlled access
- Cleaner code

Abstraction (ABC Module)

What is Abstraction?

- Hide **implementation**
- Show only **what is needed**



Real-World Example

ATM:

- You press buttons
- You don’t know internal logic

Using ABC(Abstract Base Classes) Module

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Child Class Implementation

```
class Rectangle(Shape):
    def area(self):
        print("Area = length * breadth")

r = Rectangle()
r.area()
```

Rule

- Cannot create object of abstract class
- Child must implement abstract methods

Corrected Code

Step 1: Abstract Base Class (Rule only)

- **Rule:** Any shape **must** have an area() method

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Corrected Code

Step 2: Child Class Implements Logic (Rectangle)

```
class Rectangle(Shape):
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self):
        result = self.length * self.breadth
        print("Area of Rectangle:", result)
```

Corrected Code

Step 3: Create Object & Print Area

```
r = Rectangle(10, 5)
r.area()
```

↓

```
Area of Rectangle: 50
```